2.2.8 Giving Rules for Built-in Functions

The last few sections have discussed how you can specify rules for functions that you introduce into *Mathematica*.

You can use exactly the same methods to add new transformation rules for functions that are already built in to *Mathematica*. This capability is powerful, but potentially dangerous. *Mathematica* will always follow the rules you give it. This means that if the rules you give are incorrect, then *Mathematica* will give you incorrect answers.

To avoid the possibility of changing built-in functions by mistake, *Mathematica* "protects" all built-in functions from redefinition. If you want to give a definition for a built-in function, you have to remove the protection first. After you give the definition, you should usually restore the protection, to prevent future mistakes.

`Unprotect[`*f*`]`	remove protection
`Protect[`*f*`]`	add protection

Protection for functions.

Built-in functions are usually "protected", so you cannot redefine them.

```
In[1]:= Log[7] = 2
Set::write:
    Attempt to assign to write-protected symbol Log.
Out[1]= 2
```

This removes protection for `Log`.

```
In[2]:= Unprotect[Log]
Out[2]= {Log}
```

Now you can give your own definitions for `Log`. This particular definition is not mathematically correct, but *Mathematica* will still allow you to give it.

```
In[3]:= Log[7] = 2
Out[3]= 2
```

Mathematica will use your definitions whenever it can, whether they are mathematically correct or not.

```
In[4]:= Log[7] + Log[3]
Out[4]= 2 + Log[3]
```

This removes the incorrect definition for `Log`.

```
In[5]:= Log[7] =.
```

This restores the protection for `Log`.

```
In[6]:= Protect[Log]
Out[6]= {Log}
```

You can define your own rules to add new features and capabilities to existing built-in functions.

There is no transformation rule for logarithms of products built in to *Mathematica*.

```
In[7]:= Log[a b c]

Out[7]= Log[a b c]
```

You can add the rule yourself. You need to unprotect Log before making the assignment.

```
In[8]:= (
   Unprotect[Log] ;
   Log[x_ y_] := Log[x] + Log[y] ;
   Protect[Log]
  )

Out[8]= {Log}
```

Now *Mathematica* knows how to expand out logarithms of products.

```
In[9]:= Log[a b c]

Out[9]= Log[a] + Log[b] + Log[c]
```

?Log just gives you a message about the built-in Log function.

```
In[10]:= ?Log

Log[(base:E), x] gives the logarithm of x (default is
  natural log).
```

??Log shows you all the information *Mathematica* has about Log, including the transformation rule you just added.

```
In[10]:= ??Log

Log[(base:E), x] gives the logarithm of x (default is
  natural log).
Attributes[Log] = {Listable, Protected}
Log/: Log[(x_) (y_)] := Log[x] + Log[y]
```

You can also set up your own transformation rules that override some of the rules built into *Mathematica*. *Mathematica* always tries to use your rules *before* it uses built-in ones.

There is a built-in rule for simplifying Log[Exp[*expr*]].

```
In[10]:= Log[Exp[y]]

Out[10]= y
```

You can give your own rule for Log[Exp[*expr*]].

```
In[11]:= (
   Unprotect[Log] ;
   Log[Exp[expr_]] := logexp[expr] ;
   Protect[Log] ;
 )
```

Your rule overrides the built-in one.

```
In[12]:= Log[Exp[y]]

Out[12]= logexp[y]
```

The transformation rules that are built in to *Mathematica* are intended to be

appropriate for the broadest possible range of calculations. You may find cases, however, where you do not like what the built-in rules do. In such cases, you can always give your own rules to override the ones that are built in.